CYBR-580 | Constraint Solver Modernizer

Aaron Williams-Breth

Fall 2026

Task 1: Implementation and Summary

Code Summary

The code solves Sudoku as a CSP. The variables are the 81 squares, each domain is the digits still possible for that square, and the constraints are 27 Alldiff constraints (rows, columns, boxes). `constrain` starts every square with all nine digits and assigns the clues through fill, which calls `eliminate` to remove the other digits and propagate the change. If propagation does not finish the puzzle, `search` picks the square with the fewest candidates, tries each digit on a copy of the grid, and backtracks on failure.

Connection to Class Concepts

`units` and `peers` are the constraint graph. The clues are unary constraints. `eliminate` has two rules: when a domain has one value left, remove it from every other square in its Alldiff constraint, and when a digit has only one possible square in a unit, assign it there. An empty domain returns `None`, meaning the problem is impossible. `search` is backtracking where a state is a partial assignment and an action assigns one variable. It uses Minimum-Remaining-Values, acting like its Maintaining Arc Consistency.

Task 2: Improvements and Optimization

AI was used in the research of Task 2.

Recent Techniques and Production Efficiency

I found that there are mainly two ways to speed up the process for this sudoku code, but they are not new techniques. The first way is through changing the representation of the puzzle through a bitmask representation instead of strings. Since most of the time is spent in the `elimiate` function, where it is replacing strings and dictionary lookups, we can instead change each square into a interger and therefore the entire playing field is 81 integers in a list. This makes it much cheaper to read, write, and lookup then a dictionary. The tradeoff is human comprehension as the list is harder for us to read but fine for the computer.

The other performance increase I found was to decrease the number of search nodes through increasing the strength of the pruning. Very simple in concept and implementation by including naked pairs and locked candidates (something I had to look up) helps cut down the search tree. It does come with the tradeoff of each rule costs more work per node, but the offset of less nodes to search I believe is a net improvement (only if it outweighs the time taken on hard puzzles).

For a long term production change to make this more efficent? Don't write it in Python, and instead switch to a compiled language like C, Rust, or C++. It seems like a simple cop out answer but that would bring the most efficeny improvements to the software as Python is just one of the slowest languages compared to a compiled language.

Performance Analysis and Speedup

I researched these changes but did not implement them myself. For the integer representation, the argument comes from how Python works. Strings can't be changed, so every replace builds a brand new string, and every dictionary lookup has to hash a string key like `A1`. With integers, removing a digit is one bit operation, checking "only one digit left" is one comparison, and copying the grid is a slice of a flat list of 81 numbers. Since `eliminate` runs thousands of times per puzzle, I would expect a several-fold speedup, but I can't give an exact number without testing it. Stronger pruning was not tested, so I cannot give a number, but I would expect it to help mostly on hard puzzles and very little on easy ones since each rule adds work per node. Switching to a compiled language like C, Rust or C++ should be the biggest change, and published results for python comparison tables showcase it could be 100x faster.

Optimality and Future Solutions

Since I found improvments I will not comment on how this is the optimal solution. I will also not comment on when a better solution can be found as it already has been, in the case of Tdoku which was written in C++/C. I am under the assumption that any future gains will not be any major improvements compared to the times show in Tdoku.

Task 3: Readability and Maintainability

Structural Readability

The structure is easy to follow. It is split into labeled parts, the functions are short with docstrings, and the names (`peers`, `units`, `eliminate`) are straighforward. The more abstract and math heavy stuff took a little bit more parsing and looking up python docs to understand but its nothing too cmoplicated that I spent more then an hour reading and writing. The code I rewrote helped in this process a lot, as it walked me through the thought process along with reading the documentation within the provided link to the code.

Team Integration

If I was in the position to work on this codebase and extend it, I would do so. The code is not too complex and its quite small in scope to some projects I have worked on (CSCD-350) that I would not have trouble integrating myself into the team to extend the codebase. The real question would be what to extend it to, and to actually play a game of Sudoku so I understand the rules to be able to expand the CSP solution for this program.

Task 4: Cybersecurity Applications as CSPs

Definition of a CSP: `A mathematical problem where a set of variables must be assigned values that satisfy a specific set of rules or limitations.`

Application 1: Access Control for Corporate Networks

Problem and Formulation Access control is a huge portion of how security is handled (which users and devices may connect to which servers, ports and network segments). In a CSP format each access decision (user group, resource) is a variable with the domain of {allow, deny}. For example a unary constraint is "guests cannot reach the database" while "a web server must reach its backend" is a binary constraint, and "all management ports use the same policy" is a global constraint.

Solver Efficiency and Weaknesses With the scope of the problem the only "large" part of it is the size and amount of groups, not the amount of possible states. The domains are tiny {allow, deny} and most constraints are unary or binary, so node consistency and Arc Consistency prune most of the space before any search, and the solver can check thousands of access decisions faster than a person can. The main weakness like I said above is the number of variables (ports, user groups, etc), making it a strong k-consistency which

is exponential in time and space. It is limited like most CSP's where the crafted policies done wrong or not implemented is something the solver cannot fix on its own.

Application 2: Application Version Hardening Checklist for Blue Team

Problem and Formulation Active blue team hardening is something very current in my life and I thought this would work perfectly in a CSP. A blue team needs a hardened baseline for each host, meaning which version of each application, library and OS component to run and which settings to turn on. A CSP format for each component is a variable with the domain of its available versions or settings. An example of a unary constraint is removing all versions of software with a CVE, while a binary constraint would be maintaintg a certain version of a library for a web application, then a global would be having a single package version per host in a network.

Solver Efficiency and Weaknesses Like the Access Control CSP, the domains are small, as software version's can only be so many (100 versions max for most software), allowing us to use node consistency to prune the vulnurable versions right away without worry. Then Arc Consistency can be applied to versions with partner, leaving not much of the tree left for the search to do. Another weakness that is related to the data given, as it can only accomplish its pruning by knowing the CVE's associated with a version of software, if the data is poisened or not updated, the solution gives back a safe checklist that isnt truly safe. It is also NP complete, so very large dependency graphs can still be exponential, and if every allowed version is vulnerable there is no solution at all.

Application 3: Moving Target Defense: Port Hopping (Shoutout CYBR-410)

Problem and Formulation The entire point of MTD is changing the attack surface of a machine, to be more defensible through removing the advantage of an attacker's recon (Port Hopping). In CSP terms, each service is a variable with its domain being ports/addresses. A unary constraint is removing the port the service is on right now (and the last few it used, so its never in a possible loop of two ports) so every hop lands somewhere new, and also removing ports that are reserved or blocked. A binary constraint is that the server and its legitimate clients must agree on the same port at the same time. An Alldiff global constraint is that no two services can share a port, and an Atmost constraint limits how many services can hop at once so the machine does not drop everything together.

Solver Efficiency and Weaknesses The domains are small (at most 65k ports, and far fewer after application of the unary constraints), so node consistency and the Alldiff procedure from class prune the options right away and the search barely backtracks. The main weakness for the CSP is time, since the solve has to finish before the next rotation of ports, and since the CSP is NP-complete a large network may not because of the exponential complexity. I would assume an attacker could also apply and find their own constraints after enough time to replicate the progagation to shrink down their own search for the port.